

SUPSI

Valutazione di WebGPU per applicazioni di Realtà Virtuale tramite integrazione con OpenXR/OpenVR

Studente/i

Mazza Luca

Relatore

Peternier Achille

Correlatore

Paoliello Marco

Committente

Peternier Achille

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11287

Anno

2025 / 2026

Data

7 agosto 2026

Vorrei qui ringraziare

Indice

1	Abstract	1
2	Introduzione	3
2.1	Pianificazione	4
2.2	Requisiti	5
3	Stato dell'arte	7
3.1	Standard didattico - OpenGL & OpenVR	8
3.2	Frammentazione - Vulkan, DirectX & Metal	8
3.3	Comparativa - OpenVR & OpenXR	9
3.4	L'emergere di WebGPU	10
3.5	Conclusione	11
4	Design e implementazione	13
4.1	WebGPU	14
4.1.1	Architettura	14
4.1.2	Inizializzazione contesto & Windowing System	15
4.1.3	Gestione Memoria e assets	17
4.1.4	Pipeline e Bind Group	17
4.1.5	Command Encoder e Render Loop	18
4.2	Interoperabilità wgpu-native+OpenXR	18
4.2.1	Rust <i>FFI Injection</i>	19
4.2.2	Estensioni di Vulkan	20
4.3	Transizione a DirectX12	21
4.3.1	Criticità del Backend Vulkan	21
4.3.2	Estrazione Handle D3D12 e ownership in wgpu-native	21
4.3.3	Sincronizzazione e composizione	23
4.4	Altri tentativi	25
4.4.1	Passaggio a Dawn	25
4.4.2	Inversion of Control (IoC)	27
4.5	Conclusione	29
5	Test e validazione	31
5.1	Complessità e Overhead di Sviluppo	32
5.2	Performance	32
5.2.1	VRidge + iPhone	32
6	Risultati	37
6.1	Manuale d'uso	37
7	Conclusioni	38
7.1	Sviluppi futuri	38

Glossario	I
Sitografia	IV

Elenco delle figure

3.1	WebGPU Design	10
3.2	OpenXR vs OpenVR (Khronos Group)	11
4.1	Shader class	14
4.2	WgpuBuffer class	14
4.3	Pezzi mancanti per WebGPU + OpenXR	18
4.4	Approccio <i>Inversion of Control</i>	27
5.1	Frame "Droppati" (<i>VRidge</i>)	35

Elenco delle tabelle

2.1	Requisiti del progetto	5
5.1	Test FPS (<i>VRRidge</i>)	33
5.2	Test Frame Time (<i>VRidge</i>)	33
5.3	Test Utilizzo CPU (<i>VRidge</i>)	34
5.4	Test Utilizzo RAM (<i>VRidge</i>)	34
5.5	Comparativa finale (<i>VRidge</i>)	35

Elenco dei codici sorgente

1	Inizializzazione contesto finestra (Windows)	15
2	Inizializzazione contesto finestra (Linux Wayland/X11)	15
3	Inizializzazione contesto finestra (macOS)	16
4	Generazione dei dati texture con padding	17
5	Esempio di allineamento a 16 bytes	17
6	Esempio di FFI in Rust	19
7	Estrazione di una VkImage	19
8	Abilitazione estensioni di Vulkan	21
9	Reference Counting per il ritorno di Handle nativi	22
10	Funzione di recupero degli handle D3D12	22
11	Assegnazione <i>Non-Owning</i> di puntatori al Graphics Binding	22
12	Costruzione dinamica della matrice di View tramite dati OpenXR	24
13	Gestione degli stati di memoria tramite D3D12 Resource Barriers	24
14	Costruttore inaccessibile per ExternalImageExportInfoVK	26
15	Enumerativo ExternalImageType	28

Capitolo 1

Abstract

Capitolo 2

Introduzione

Negli ultimi anni, l'evoluzione delle API grafiche ha portato ad un progressivo allontanamento dagli standard tradizionali, come OpenGL¹, in favore di soluzioni più moderne, che forniscono più controllo direttamente sull'hardware delle unità grafiche dei calcolatori. In questo panorama, WebGPU² si è imposta non solo come nuova generazione di API grafiche per il Web, ma anche come una solida alternativa per contesti nativi, grazie ai kit di sviluppo che la rendono compatibile con diversi linguaggi a basso livello, come C++ e Rust. Un progetto precedentemente sviluppato ha già analizzato e dimostrato l'efficacia della transizione da OpenGL a WebGPU per corsi introduttivi alla computer grafica.

Lo scopo del presente progetto è di estendere tale studio di fattibilità in un settore con ancora maggiori esigenze, ovvero il dominio della *Realtà Virtuale* (VR). L'obiettivo principale è la valutazione dell'impiego di WebGPU in un ambiente nativo C/C++, e che questo possa rappresentare una valida alternativa ad OpenGL per lo sviluppo di applicazioni VR, investigando la complessità e l'efficacia della sua integrazione con framework standard quali OpenXR³ e OpenVR⁴.

Come per quanto riguarda i progetti precedentemente sviluppati, per valutare correttamente questa tecnologia è fondamentale tener sempre presente il contesto accademico in cui si andrà ad operare. Questo lavoro si rivolge in primis al modulo opzionale M-I6331Z - *Realtà Virtuale* della SUPSI, che si basa sulle conoscenze acquisite nel corso obbligatorio M-I5203 - *Grafica*. Lo scopo di tale corso è di insegnare allo studente lo sviluppo di un'applicazione interattiva in C++, interfacciandosi direttamente con le API di basso livello, senza affidarsi a game engine preconfezionati.

In questo scenario, la semplicità d'utilizzo è cruciale nel determinare la fattibilità; una soluzione troppo complessa rischierebbe di spostare inavvertitamente il focus del corso. Gli studenti e il docente si ritroverebbero a dedicare troppo tempo a districarsi tra le difficoltà di implementazione dell'API grafica, sottraendolo all'apprendimento dei concetti fondamentali della *Realtà Virtuale*. Tuttavia, lo sviluppo VR impone requisiti stringenti in termini di latenza, prestazioni e gestione diretta dell'hardware, elementi dove le API moderne come WebGPU eccellono, grazie al loro paradigma basato su pipeline esplicite.

Durante il progetto verrà pertanto sviluppato un prototipo funzionante che utilizzi WebGPU come backend grafico per il rendering di una scena VR stereoscopica. Verrà creato un layer di integrazione tra WebGPU e le API VR, affrontando e risolvendo le sfide tecniche legate all'acquisizione delle pose dei dispositivi, alla gestione delle swapchains e alla rigorosa sincronizzazione dei frame richiesta dal runtime, come ad esempio SteamVR.

¹<https://opengl.org>

²<https://webgpu.org>

³<https://khronos.org/openxr>

⁴<https://github.com/ValveSoftware/openvr>

L'esito di questa implementazione permetterà infine di confrontare l'approccio WebGPU con le soluzioni tradizionali, misurando in modo oggettivo il livello di effort richiesto per lo sviluppo e le performance ottenute. I risultati raccolti forniranno delle linee guida concrete per determinare l'effettiva fattibilità e le modalità ottimali per l'adozione di WebGPU in ambito didattico e applicativo nel contesto della Realtà Virtuale.

Con questo documento non si intende però produrre una guida dettagliata per quanto riguarda l'API di WebGPU, e nemmeno all'utilizzo di OpenXR o alle funzionalità VR, bensì saranno espliciti solamente i concetti essenziali alla comprensione di questo lavoro. Se si desidera apprendere i concetti riguardanti entrambe le API menzionate si può fare riferimento alla guida per i fondamentali di WebGPU[1] e a quella di OpenXR[2].

È anche da notare che questo documento non ha lo scopo di rendere noto ogni singolo dettaglio di codice sviluppato, bensì di accompagnare alla comprensione del problema, dei passi intrapresi per cercare di risolverlo e, se finalmente è fattibile sostituire l'approccio tradizionale con WebGPU.

2.1 Pianificazione

Di seguito sono descritte le differenti fasi atte allo sviluppo del progetto. In quanto la natura del progetto sia sperimentale, ed infine l'obiettivo sia quello di verificare la fattibilità di utilizzo, il piano di lavoro non eccede nei dettagli o nei vincoli ma risulta completo, garantendo la flessibilità necessaria e l'immediata risposta ad eventuali problemi o deviazioni dai mezzi discussi inizialmente.

1. **Analisi WebGPU:** Analizzare l'approccio necessario per lo sviluppo di un'applicazione grafica facente uso dell'API di WebGPU.
2. **Analisi OpenXR:** Analizzare l'approccio utilizzato nei progetti precedenti (specialmente per quanto riguarda *XRBridge*⁵) per lo sviluppo di un'applicazione grafica che faccia utilizzo di OpenXR per fornire funzionalità per la realtà virtuale.
3. **Demo WebGPU indipendente:** Derivante dall'analisi riguardante l'API di WebGPU, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità dell'API WebGPU. Questa demo deve rimanere semplice e ridurre la complessità di integrazione di ulteriori funzionalità, specialmente quelle riguardanti la realtà virtuale.
4. **Demo OpenXR indipendente:** Derivante dall'analisi riguardante OpenXR, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità di realtà virtuale. Questa demo deve essere imperativamente concentrata sui requisiti del corso di realtà virtuale e concentrarsi ad adattare unicamente ciò che è necessario per il corso. Ulteriormente, questa deve essere integrabile nell'ambiente WebGPU, deve quindi avere un approccio modulare, il quale permetterà di rimuovere le funzionalità OpenGL e sostituirle con quelle di WebGPU.
5. **Swapchain condivisa:** Sviluppare una swapchain, condivisa tra WebGPU e OpenXR, che permetta di condividere le texture da mostrare tramite gli occhi dell'headset, tra l'API WebGPU e quella di OpenXR. Ciò si riduce nella produzione di una libreria bridge, che utilizzi uno dei "minimi comun denominatori"⁶ tra le due tecnologie, in questo caso una tra Vulkan, DirectX o Metal.

⁵<https://github.com/JollyCookie2000/xr-bridge>

⁶WebGPU si interfaccia direttamente con l'API di sistema per la rappresentazione nativa di immagini grafiche.

6. **Sincronizzazione & Rendering Stereoscopico:** Integrare la sincronizzazione del loop di rendering di WebGPU con i tempi dettati da OpenXR. Deve essere inoltre modificata la pipeline di WebGPU per renderizzare la scena due volte, basandosi sulle matrici di *projection* e *view* fornite da OpenXR per occhio sinistro e destro.
7. **Demo unificata:** Produrre una demo che unisca tutte le funzionalità richieste dal corso di Realtà Virtuale, utilizzando l'architettura prodotta nelle fasi precedenti. Questa deve dimostrare in maniera semplice e efficace i risultati ottenibili dalla combinazione di WebGPU e OpenXR.
8. **Benchmarking:** Misurare il Framerate (FPS) e la latenza della demo, utilizzando appropriati strumenti di misurazione.
9. **Valutazione effort:** Confrontare la pipeline prodotta con quelle utilizzate negli anni passati del corso; valutare le principali differenze e le difficoltà di comprensione, la verbosità e l'ostacolo di debug.
10. **Linee guida conclusive:** Produrre una guida finale che mostri brevemente le conclusioni tratte durante lo sviluppo e che guidi l'eventuale integrazione della pipeline nel corso.

Essendo questo uno studio di fattibilità esplorativo, la pianificazione ha mantenuto un approccio iterativo. Come si vedrà nel capitolo 4, le rigidità architetturali incontrate richiedono dei "pivot" durante lo sviluppo, forzando la deviazione dal piano originale, verso l'esplorazione di pattern architetturali differenti.

È anche necessario notare che sebbene ovvio, nel caso l'applicazione dei concetti non risultasse fattibile proprio dal punto di vista tecnico e non accademico, la valutazione dell'effort e le eventuali analisi delle performance risultano poco utili e tralasciabili.

Questo rapporto è stato scritto durante tutta la durata del progetto, parallelamente a tutte le fasi descritte sopra.

2.2 Requisiti

Tabella 2.1: Requisiti del progetto

ID	Descrizione	Note
R-00	Deve presentare un backend WebGPU, con <code>wgpu_native</code> , in C++	< C++20
R-01	Deve presentare l'integrazione di OpenXR per gestire funzionalità VR/XR	-
R-02	Deve acquisire i dati di tracciamento dell'HMD	-
R-03	Deve gestire allocazione texture e submission duale con swapchains	-
R-04	Deve gestire la sincronizzazione con il framerate runtime VR	-
R-05	Deve presentare una demo completa di scena 3D, che utilizzi WebGPU	-
R-06	Deve essere compatibile con Windows e Linux	-
R-07	Deve includere analisi complessità, comparata alla soluzione OpenGL	-
R-08	Deve includere un'analisi della performance (benchmark)	-
R-09	Deve includere un verdetto finale e guida all'utilizzo	-

Capitolo 3

Stato dell'arte

Negli ultimi dieci anni, l'ecosistema dello sviluppo di applicazioni grafiche interattive ha visto intercorrere una trasformazione radicale. L'industria si è pian piano allontanata dai paradigmi tradizionali, basati su macchine a stati globali, per abbracciare interfacce di programmazione dal carattere esplicito, progettate per rispecchiare fedelmente l'architettura delle GPU multi-core. Ciò ha portato le API, contrariamente al trend seguito dalla maggior parte dell'ingegneria del software che si muove verso API di alto livello e astrazioni più vicine allo sviluppatore, a scendere sempre più di livello, avvicinandosi alla programmazione diretta dell'hardware grafico.

Parallelamente, il settore della Realtà Virtuale e Aumentata, comunemente raggruppato sotto il termine *Spatial computing* o XR, è maturato, passando da prototipi frammentari guidati da singoli vendor a standard aperti e unificati. Soprattutto per questo settore, il mercato ha avuto un moderato picco durante il periodo tra il 2020 ed il 2021, il quale ha poi visto un rapido declino, dovuto a diversi fattori, ma principalmente a scelte e underperformance dei produttori; specialmente lo spostamento dei settori di ricerca e sviluppo dei maggiori esponenti, come Microsoft e Meta, nel settore dell'Intelligenza Artificiale, in maniera affrettata (e poco considerata, visti i risultati ad oggi) ha posto fine all'evoluzione dei prodotti commerciali. Anche se il rilascio di un nuovo prodotto Apple nel settore, il prezzo imposto dalla casa di Cupertino non ha aiutato il mercato a rendere la tecnologia attrattiva.

Questo capitolo analizza le tecnologie attualmente impiegate in ambito didattico e industriale, evidenziando la crescente trasformazione dell'ecosistema grafico, le limitazioni e "l'invecchiamento" degli approcci tradizionali e la transizione architetturale verso nuovi standard aperti, come OpenXR e WebGPU.

3.1 Standard didattico - OpenGL & OpenVR

Nel contesto accademico e della formazione in computer grafica, il corso di Realtà Virtuale ha fatto affidamento sull'utilizzo di OpenGL per il rendering e OpenVR per l'interfacciamento con gli HMD.

OpenGL¹, introdotto nel 1992, è un'API implicita basata su una macchina a stati finiti. Il suo successo decennale, specialmente nell'ambito accademico, deriva dalla curva di apprendimento che risulta relativamente dolce; l'astrazione di alto livello utilizzata dagli sviluppatori risulta semplice, in quanto nasconde al programmatore la complessa gestione della memoria video, la sincronizzazione dei thread e la sottomissione dei comandi alla GPU. Tuttavia, questa astrazione ha un costo architetturale severo: i driver di OpenGL devono eseguire una massiccia mole di euristiche in background per tradurre le chiamate high level in istruzioni comprensibili per la scheda grafica, causando un notevole driver overhead e un'imprevedibilità delle prestazioni, soprattutto del frame stuttering, difetti critici nell'ambito della realtà virtuale, dove la latenza *motion-to-photon* deve rimanere strettamente inferiore ai 20 millisecondi.

OpenVR², sviluppato da Valve come SDK per la piattaforma SteamVR, è stata la prima API di successo commerciale a standardizzare l'accesso a device dedicati alla realtà virtuale *PC-VR*. Nata in un periodo di forte sperimentazione, questa tecnologia è intrinsecamente legata all'ecosistema di Valve, soprattutto con i visori di produzione HTC, per i quali le due aziende hanno collaborato nella produzione. Si basa su un modello nel quale l'hardware sta al centro dell'attenzione; infatti il runtime costringe l'hardware concorrente a tradurre i propri dati per renderli compatibili con il formato di un HTC Vive³, per esempio. Questa soluzione costringerebbe il programmatore a rilasciare una nuova patch ogni volta che un nuovo controller, visore o elemento periferico per il sistema VR viene rilasciato. Pur offrendo un'integrazione diretta e robusta con OpenGL, al momento si tratta di una tecnologia legacy, non più allineata con la traiettoria dell'industria moderna.

3.2 Frammentazione - Vulkan, DirectX & Metal

Il limite fisiologico di OpenGL ha portato alla nascita di una nuova generazione di API grafiche di basso livello; il successore naturale è **Vulkan**, sviluppato dallo stesso Khronos Group, **DirectX**, sviluppato da Microsoft con l'obiettivo di spingere il Gaming su PC, e **Metal**, di produzione Apple, per tutte le piattaforme correntemente sul mercato (iOS, macOS, tvOS, ecc...). Sebbene queste API garantiscano prestazioni eccezionali eliminando l'overhead dei driver e permettendo il multithreading nativo, hanno frammentato drasticamente il mercato, rendendo lo sviluppo multipiattaforma un'impresa titanica.

Vi sono due fattori critici che rendono queste API problematiche per la didattica:

1. Vulkan richiede un approccio esplicito alla gestione delle risorse. Il developer deve allocare manualmente la memoria, gestire i descriptor set, configurare le pipeline memory barrier e orchestrare le code di comandi. Realizzare il rendering di un singolo triangolo colorato utilizzando Vulkan richiede la stesura di 1000-1500 righe di codice, che a confronto delle 50-100 di OpenGL risultano eccessive per un corso di dieci settimane. Inoltre, un corso accademico focalizzato sulla realtà virtuale, dove gli argomenti principali dovrebbero essere la stereoscopia, l'interazione spaziale e la cinematica, forzare gli studenti ad apprendere quest'API significherebbe consumare l'intero semestre solo per inizializzare l'ambiente grafico.

¹<https://opengl.org>

²<https://github.com/ValveSoftware/openvr>

³<https://vive.com>

2. Apple ha ufficialmente deprecato OpenGL sui propri sistemi operativi, per poi rimuovere completamente il supporto nativo a tutte quelle tecnologie non-proprietarie che compongono lo standard *de-facto* dell'industria. Attualmente infatti, l'unica API grafica nativa supportata dall'hardware Apple Silicon e dai sistemi operativi moderni della casa californiana, è **Metal**. Apple ha rifiutato di adottare Vulkan, costringendo gli sviluppatori ad affidarsi a complessi layer di traduzione, come *MoltenVK*, per ottenere una parvenza di compatibilità. Questa tendenza a murare il proprio giardino distrugge il paradigma *Write once, Compile everywhere*, rendendo impossibile fornire un ambiente di sviluppo nativo e condiviso tra chi sceglie di utilizzare Windows, Linux o macOS.

3.3 Comparativa - OpenVR & OpenXR

La medesima frammentazione vissuta nel mercato delle API si è ripercossa su quello dell'hardware VR. Per risolvere questo problema, il *Khronos Group*⁴ ha rilasciato OpenXR, l'attuale standard industriale open-source per lo spatial computing, adottato universalmente da tutti i principali produttori, inclusa Valve stessa, che ha attivamente incoraggiato l'abbandono di OpenVR⁵.

Le differenze architetturali tra le due API sono profonde e rappresentano una rivoluzione concettuale nel modo in cui vengono concepite le applicazioni immersive. Di seguito le principali divergenze

- **Gestione Input (Hardware-Centric/Action-Based):** in OpenVR, il programmatore interroga direttamente l'hardware; questo approccio lo obbliga a riscrivere il codice ogni volta che un nuovo visore o una nuova periferica viene rilasciata sul mercato. In risposta a questo problema, OpenXR introduce un sistema **Action-Based**. Qui, lo sviluppatore definisce in modo astratto delle azioni logiche, per poi passarle al Runtime, il quale si occupa di mapparle sull'hardware utilizzato correntemente dall'utente in base a profili di interazione. Ciò rende le applicazioni scritte oggi con hardware che verrà rilasciato anche nel lontano futuro, senza necessità di modifiche o patching.
- **Gestione Spazi/Tracking:** OpenVR restituisce le matrici di posizionamento (*Poses*) basate su un sistema di coordinate cablato. OpenXR invece introduce il concetto di *XrSpace*⁶, dove ogni elemento, che si tratti del visore, del controller o della zona di gioco, è codificato come "Spazio". Al suo interno, la posizione di un oggetto viene calcolata interrogando il runtime, chiedendogli di localizzare uno spazio rispetto ad un altro, provvedendo a fornire anche stime sulle velocità lineari e angolari basate su complessi algoritmi di predizione, oltre che la posizione degli oggetti.
- **Architettura Swapchain:** per quanto riguarda OpenVR, l'applicazione grafica ha il compito di generare le texture e di provvederle al visore, tramite la funzione *Submit*. Al contrario, per quanto riguarda OpenXR, l'architettura è pensata all'inverso; è il runtime a possedere e allocare le immagini della Swapchain, garantendo ottimizzazione per l'hardware. L'applicazione deve quindi implementare un Graphic Binding esplicito, richiedendo al runtime i puntatori di memoria per poi disegnarci sopra. L'inversione del controllo rende estremamente complessa l'integrazione di API grafiche non ufficialmente supportate.

⁴<https://khronos.org>

⁵<https://windowscentral.com/steamvr-gets-open-source-upgrade-valve-transitions-openxr-api>

⁶<https://registry.khronos.org/OpenXR/specs/1.1/man/html/XrSpace.html>

3.4 L'emergere di WebGPU

Per colmare il vuoto lasciato dalla deprecazione di OpenGL e dalla complessità proibitiva di Vulkan, il *World Wide Web Consortium* (W3C) ha sviluppato WebGPU. Nata originariamente per succedere a WebGL, per i browser web, questa tecnologia è progettata con lo scopo di esporre le capacità delle API moderne garantendo sicurezza, portabilità ed un livello di astrazione ergonomico per lo sviluppatore.

La vera rivoluzione per l'ambiente desktop, ed il fulcro tecnologico di questo progetto, è il porting nativo di questa API tramite implementazioni C/C++, come **wgpu-native** e **Dawn**, sviluppate rispettivamente da Mozilla, in Rust e da Google in C++. Queste librerie operano come *Render Hardware Interface* (RHI) universale; lo sviluppatore scrive il codice relativo alla propria applicazione grafica, utilizzando l'API WebGPU standard e scrive gli shader nel linguaggio WGSL (WebGPU Shading Language). Il backend esegue poi automaticamente la traduzione *just-in-time* e invia i comandi all'API nativa del sistema operativo, quindi Vulkan/DirectX per quanto riguarda Windows, Vulkan per quanto riguarda Linux ed infine Metal per macOS.

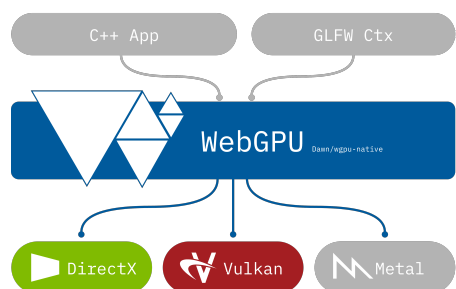


Figura 3.1: WebGPU Design

WebGPU nativo risolve quindi simultaneamente due problemi fondamentali: riduce il boilerplate necessario per controllare l'hardware grafico, sempre utilizzando le API grafiche più moderne, performanti, efficienti e ottimizzate, e garantisce una compatibilità cross-platform assoluta, aggirando le restrizioni e l'isolazionismo di Apple.

Tuttavia, questa sicurezza e portabilità derivano da un rigoroso sandboxing dell'API; mentre le API grafiche tradizionali permettono facilmente la condivisione di risorse inter-process, un requisito fondamentale per lavorare in contesti di realtà virtuale dove il computer ed il visore devono scambiarsi fotogrammi senza costi aggiuntivi di memoria, WebGPU sembra nascondere tali funzionalità per mantenere la sicurezza alta. Questo scontro di paradigmi rappresenta la sfida principale.

3.5 Conclusione

L'industria dello spatial computing è ormai stabilmente posizionata su OpenXR, per la logica XR.

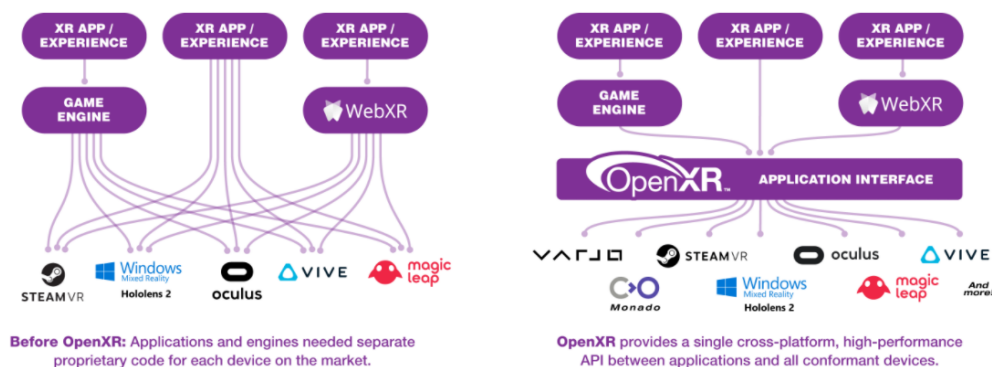


Figura 3.2: OpenXR vs OpenVR (Khronos Group)

Sul fronte grafico, mentre i motori commerciali (Unity, Unreal) o quelli proprietari, gestiscono le complesse API grafiche internamente, per lo sviluppo nativo ed educativo, WebGPU si sta affermando come soluzione più promettente.

Tuttavia, esiste attualmente un vuoto tecnologico per quanto riguarda l'API del W3C e OpenXR; tra le due tecnologie non esiste alcun Graphic Binding ufficiale da parte di Khronos. Quest'ultimo infatti definisce binding per OpenGL, Vulkan, Direct3D e Metal, ma non per WebGPU, essendo quest'ultima un'RHI e non consistente in un driver nativo.

Il presente lavoro si inserisce esattamente in questo contesto; l'obiettivo tecnico e scientifico è di dimostrare la fattibilità di un layer di interoperabilità che permetta di implementare nativamente con `wgpu_native` e che questa implementazione possa comunicare con la complessa swapchain ed il frame-pacing di OpenXR. Ciò permetterebbe di pensionare lo stack legacy e sostituirlo con uno all'avanguardia per il corso, con completa sostenibilità didattica e cross-platform per integrare i calcolatori di ogni studente.

Capitolo 4

Design e implementazione

Questo capitolo descrive in dettaglio la fase di implementazione del motore di rendering, che funge da *Proof of concept* per determinare la fattibilità dell'architettura sotto esame. L'implementazione viene affrontata in modo incrementale, data la mancanza di esperienza per quanto riguarda le tecnologie in questione, e viene divisa in tre pilastri fondamentali, che verranno esplorati nelle sezioni seguenti:

1. Il motore grafico di base WebGPU, consistente nella costruzione di una pipeline di rendering esplicita ed autonoma, utilizzando l'API `wgpu_native`. Questa fase risolve le critiche relative all'allineamento della memoria, alla gestione esplicita dello stato ed al caricamento degli assets.
2. L'integrazione di OpenXR, che vede l'implementazione di un'interfaccia indipendente dall'hardware per comunicare con i visori di realtà virtuale, gestendo il tracciamento spaziale, gli stati della sessione e l'acquisizione della swapchain.
3. Il ciclo applicativo principale che fa da ponte tra i due sottosistemi, sincronizzando il rendering stereoscopico di WebGPU con i requisiti temporali di visualizzazione del compositor OpenXR.

Analizzando queste tre fasi, questo capitolo illustra come i requisiti ingegneristici teorici vengono tradotti in un prototipo funzionale, ottimizzato e possibilmente multiplatforma.

4.1 WebGPU

La seguente sezione si concentra sulle funzionalità di WebGPU implementate, utili a raggiungere lo stesso risultato dimostrato nel tutorial ufficiale del corso di realtà virtuale.

È importante che la codebase, sebbene facente uso di un'API con struttura e filosofia radicalmente differenti, sia strutturata il più simile possibile a quella del corso, per rendere la comparazione il più semplice ed efficiente possibile.

Altresì importante è tenere a mente che il codice prodotto è pensato per uno studente, è quindi da considerare le piattaforme e il livello di confidenza nell'apprendere un'API del genere.

4.1.1 Architettura

A differenza di OpenGL, il quale mantiene uno stato globale e gestisce la sincronizzazione della memoria in background, la nuova architettura richiede definizioni esplicite per ogni singola fase della pipeline. Per mantenere una minima leggibilità e separare qualche responsabilità, pur lasciando la struttura simile a quella del tutorial del corso per facilitarne la comparazione; per questo, seguendo l'architettura di corso, sono state modularizzate le classi:

- **Shader:** Gestisce il caricamento, la compilazione ed il ciclo di vita degli Shader Modules WebGPU, partendo dalla sorgente di codice, nel linguaggio WGSL (Web GPU Shading Language). Evita il caricamento di file esterni direttamente dal punto d'entrata del motore.

Shader
-m_module: WGPUShaderModule
+Shader() +~Shader() +loadFromFile(WGPUDevice, char*): bool +destroy(): void +getModule(): WGPUShaderModule

Figura 4.1: Shader class

- **WgpuBuffer:** Incapsula la creazione e l'aggiornamento dei buffer GPU, detti `WGPUBuffer`, imponendo il rigoroso allineamento della memoria a 4 byte, richiesto dalle specifiche dell'API per i vertici ed i dati *uniform*.

WgpuBuffer
-m_buffer: WGPUBuffer -m_alignedSize: size_t
+WGPUBuffer() +~WGPUBuffer() +create(WGPUDevice, WGPUQueue, void*, size_t, WGPUBufferUsage): bool +update(WGPUQueue, void*, size_t, size_t): void +get(): WGPUBuffer +size(): size_t

Figura 4.2: WgpuBuffer class

Questo incapsulamento garantisce che il ciclo principale dell'applicazione, includendo i descrittori della pipeline e la codifica dei comandi, rimanga pulito e focalizzato esclusivamente sull'inizializzazione e l'orchestrazione.

4.1.2 Inizializzazione contesto & Windowing System

La fase di inizializzazione del contesto, richiede il collegamento dell'istanza WebGPU ad un sistema di windowing al livello del sistema operativo. Questo requisito risulta relativamente complesso per l'integrazione della nuova API, in quanto questa è pensata per il web, quindi non avendo alcun accesso diretto alle API di sistema per l'acquisizione della finestra.

Windows

Per i sistemi operativi della famiglia Windows, l'integrazione risulta relativamente lineare, grazie all'accesso diretto alle API Win32 fornite dalla libreria GLFW. Il descrittore specifico per questa piattaforma, `WGPUSurfaceSourceWindowsHWND`, richiede la provvigione di due parametri fondamentali: l'handle dell'istanza dell'applicazione (`hinstance`), recuperabile tramite la chiamata di sistema `GetModuleHandle`, e l'handle della finestra nativa `hwnd`. Come mostrato nel listing 1, questi dati vengono incapsulati e concatenati direttamente al descrittore generico della *Surface*, sfruttando il meccansimo del puntatore `nextInChain`.

```
WGPUSurfaceDescriptor surfaceDesc= {};
#ifdef _WIN32
WGPUSurfaceSourceWindowsHWND hwndDesc = {
    .chain = { .sType = WGPUType_SurfaceSourceWindowsHWND },
    .hinstance = GetModuleHandle(NULL),
    .hwnd = glfwGetWin32Window(window)
};
surfaceDesc.nextInChain = (WGPUChainedStruct*)&hwndDesc;
```

Listing 1: Inizializzazione contesto finestra (Windows)

Linux

Nel panorama Linux, l'acquisizione del contesto della finestra richiede un approccio dinamico a causa della coesistenza di due differenti protocolli per server grafici: l'ormai legacy *X11* ed il più moderno *Wayland*. Per garantire la massima portabilità e compatibilità, l'implementazione non assume a priori l'infrastruttura sottostante, ma interroga la variabile d'ambiente `XDG_SESSION_TYPE` del sistema operativo per determinare il compositor attualmente in uso. A seconda del risultato (vedi listing 2), l'algoritmo istanzia il descrittore appropriato estraendo da GLFW i rispettivi puntatori nativi per il display e per la superficie di rendering.

```
#elif defined (__linux__)
char *envSession = std::getenv("XDG_SESSION_TYPE");
std::string windowingSystem = envSession ? envSession : "x11";
if (windowingSystem == "x11") {
    WGPUSurfaceSourceXlibWindow x11Desc = {
        .chain = { .sType = WGPUType_SurfaceSourceXlibWindow },
        .display = glfwGetX11Display(),
        .window = glfwGetX11Window(window) };
    surfaceDesc.nextInChain = (WGPUChainedStruct*)&x11Desc;
} else if (windowingSystem == "wayland") {
    WGPUSurfaceSourceWaylandSurface waylandDesc = {
        .chain = { .sType = WGPUType_SurfaceSourceWaylandSurface },
        .display = glfwGetWaylandDisplay(),
        .surface = glfwGetWaylandWindow(window) };
    surfaceDesc.nextInChain = (WGPUChainedStruct*)&waylandDesc;
}
```

Listing 2: Inizializzazione contesto finestra (Linux Wayland/X11)

macOS

L'ecosistema Apple presenta il livello di complessità più alto per quanto riguarda la creazione della superficie, poiché su queste piattaforme WebGPU si appoggia nativamente al backend grafico Metal. Per evitare di dover introdurre nella codebase C++ delle sorgenti scritte in ObjectiveC, che richiederebbe l'uso di sorgenti .mm, l'implementazione sfrutta l'iniezione a runtime tramite la funzione `objc_msgSend` che permette di interagire direttamente con le librerie di sistema Cocoa. Il processo, consultabile nel listing 3, si occupa di estrarre dapprima l'oggetto `NSWindow`, ne recupera poi la vista principale `contentView` e vi aggancia dinamicamente un livello di rendering dedicato, chiamato `CAMetalLayer`. Inoltre, durante questa fase, viene esplicitamente disabilitata la proprietà `displaySyncEnabled` del layer Metal; questo è un passaggio fondamentale per aggirare la sincronizzazione verticale forzata in modo aggressivo dal *Window Server* di macOS, permettendo all'applicazione di sbloccare il framerate per una migliore profilazione delle performance. Il livello così configurato viene finalmente passato al descrittore `WGPUSurfaceSourceMetalLayer`.

```
#elif defined(__APPLE__)
#include <objc/message.h>
#include <objc/runtime.h>
id nsWindow = glfwGetCocoaWindow(window);
id nsView = ((id*)(id, SEL))objc_msgSend(nsWindow,
    sel_registerName("contentView"));
id caMetalLayerClass = (id)objc_getClass("CAMetalLayer");
id metalLayer = ((id*)(id, SEL))objc_msgSend(caMetalLayerClass,
    sel_registerName("layer"));
((void*)(id, SEL, bool))objc_msgSend(nsView,
    sel_registerName("setWantsLayer:"), true);
((void*)(id, SEL, id))objc_msgSend(nsView,
    sel_registerName("setLayer:"), metalLayer);
((void*)(id, SEL, bool))objc_msgSend(metalLayer,
    sel_registerName("setDisplaySyncEnabled:"), false);
WGPUSurfaceSourceMetalLayer metalDesc = {
    .chain = { .sType = WGPUType_SurfaceSourceMetalLayer },
    .layer = metalLayer
};
surfaceDesc.nextInChain = (WGPUChainedStruct*)&metalDesc;
#endif
```

Listing 3: Inizializzazione contesto finestra (macOS)

Una volta configurato il descrittore specifico per la piattaforma host corrente tramite la catena di strutture `nextInChain`, la superficie viene concretamente istanziata e restituita al ciclo di vita dell'applicazione tramite la chiamata nativa dell'API.

```
return wgpuInstanceCreateSurface(instance, &surfaceDesc);
```

4.1.3 Gestione Memoria e assets

Il passaggio dal `glTexImage2D` di OpenGL al sistema adottato da WebGPU richiede il calcolo manuale degli *stride* di memoria. La libreria `stb_image` è stata integrata per caricare gli assets direttamente in un formato *top-down*, corrispondendo nativamente al sistema di coordinate delle texture WebGPU ed eliminando la necessità di invertire l'asse *y* durante il caricamento.

L'API impone un vincolo sui caricamenti delle texture tramite `wgpuQueueWriteTexture`: il parametro `bytesPerRow` deve essere obbligatoriamente un multiplo di 256 bytes. Questo problema è stato risolto calcolando e applicando matematicamente un padding al buffer dei dati grezzi dell'immagine, prima di consegnarlo alla GPU.

```
uint32_t bytesPerRow = (texWidth * 4 + 255) & ~255;
std::vector<uint8_t> paddedData(bytesPerRow * texHeight, 0)
for (int y = 0; y < texHeight; ++y) {
    memcpy(&paddedData[y*bytesPerRow], &rawData[y*texWidth*4], texWidth*4);
}
```

Listing 4: Generazione dei dati texture con padding

Un altro fondamentale cambiamento di paradigma riguarda l'allineamento delle strutture tra C++ e WGSL. Questo infatti impone rigorose regole di layout nella memoria, nelle quali tipi di dati come `vec3` devono essere strettamente allineati a limiti di 16 bytes. Per prevenire la desincronizzazione della memoria tra le `struct` di C++ a 128 bytes e la `struct` WGSL attesa, da 144 bytes, è stato introdotto un padding manuale nelle strutture dati C++ per soddisfare le aspettative di memoria della GPU, senza sprecare larghezza di banda.

```
struct LightMaterialUniforms {
    glm::vec4 lightPosition;
    glm::vec4 lightAmbient;
    glm::vec4 lightDiffuse;
    glm::vec4 lightSpecular;
    glm::vec4 matAmbient;
    glm::vec4 matDiffuse;
    glm::vec4 matSpecular;
    float matShininess;
    float padding[3];
};
```

Listing 5: Esempio di allineamento a 16 bytes

4.1.4 Pipeline e Bind Group

La pipeline di rendering funge da oggetto di stato immutabile. Il prototipo utilizza un modello di illuminazione *Blinn-Phong*, proprio come nel tutorial del corso, il tutto tradotto in una *Shader* scritta nel linguaggio WGSL. Durante la creazione della pipeline, viene sfruttata la funzionalità di *Auto-Layout*.

```
pipelineDesc.layout = nullptr;
```

Questo istruisce il backend WebGPU a inferire dinamicamente il layout del Bind Group, direttamente dalle annotazioni del linguaggio di shading (`@group` e `@binding`), riducendo significativamente la verbosità del C++.

Le risorse da fornire al programma shader sono divise in due distinti Bind Group in base alla loro frequenza di aggiornamento:

1. **Bind Group 0:** contiene gli Uniform Buffer per le matrici *Model-View-Projection* ed i parametri dei materiali di illuminazione, aggiornati ad ogni frame
2. **Bind Group 1:** contiene il *Sampler* e la *Texture View*, che rimangono statici per tutto il ciclo di vita dell'applicazione

Inoltre, le rigide regole di validazione dell'API hanno richiesto una gestione esplicita delle *C Strings*. Con i recenti aggiornamenti delle API, gli entrypoint degli shader devono essere passati come oggetti di tipo `WGPUStringView`, che abbinano il contenuto della stringa con una macro di lunghezza specifica (`WGPU_STRLEN`), piuttosto che come puntatori terminati da carattere nullo, al fine di gestire la sicurezza della memoria attraverso i confini del backend Rust.

4.1.5 Command Encoder e Render Loop

Il ciclo principale vede un `WGPUCommandEncoder` che registra una serie di istruzioni all'interno di un command buffer, che viene successivamente sottomesso alla queue della GPU in una singola operazione.

Una cruciale implementazione di stabilità all'interno del ciclo di rendering è la gestione dello stato di acquisizione della `WGPUSurfaceTexture`. Le latenze del sistema operativo, come il ridimensionamento o la riduzione ad icona della finestra potrebbero far sì che la superficie "perda" temporaneamente accesso alla texture del frame corrente; l'aggiunta di una *guard clause* che valuti lo stato `WGPUSurfaceGetCurrentTextureStatus_Success` impedisce al backend Rust di andare in *panic* alla ricezione di un puntatore nullo, garantendo la totale robustezza del motore in caso di eventi esterni.

Infine, il `WGPURenderPassDescription` richiede definizioni esplicite e complete, inclusa la specifica del parametro `depthSlice` a `WGPU_DEPTH_SLICE_UNDEFINED`, per i color attachment 2D, una configurazione necessaria per conformarsi in modo rigoroso alle specifiche di rendering delle texture 3D, recentemente introdotte in WebGPU.

4.2 Interoperabilità wgpu-native+OpenXR

L'integrazione tra WebGPU e OpenXR presenta una sfida architetturale fondamentale legata ai paradigmi di sicurezza ed astrazione delle due API; WebGPU, per sua natura, è agnostica e sicura: il suo obiettivo è di nascondere completamente i dettagli hardware sottostanti dietro ad un'interfaccia unificata. Al contrario, OpenXR per operare in ambiente VR richiede un'accesso esplicito e diretto ai puntatori nativi della GPU, ed alle allocazioni di memoria delle immagini per poter gestire la swapchain a bassa latenza.

Nella versione più recente¹, le funzioni di estrazione per il backend Vulkan sono state rimosse dall'API pubblica, sigillando di fatto i puntatori nativi all'interno dell'Hardware Abstraction Layer, scritto in Rust. Ciò rende impossibile l'inizializzazione standard di una `XrSession` basata sul contesto grafico di WebGPU.

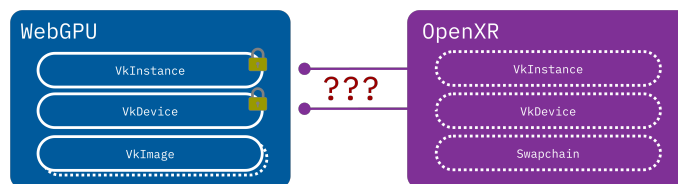


Figura 4.3: Pezzi mancanti per WebGPU + OpenXR

¹<https://github.com/gfx-rs/wgpu-native/tree/v29.0.0.0>

4.2.1 Rust FFI Injection

Per aggirare questa limitazione senza dover riscrivere interi moduli della libreria si è optato per una tecnica di iniezione di codice, tramite una tecnica chiamata *Foreign Function Interface*, direttamente nel codice sorgente di `wgpu-native`.

Sfruttando le capacità di interoperabilità e esportazione in C di Rust, vengono iniettate delle funzioni custom, in grado di interrogare l'oggetto `context` all'interno di WebGPU.

Ciò permette di forzare l'astrazione e di risalire all'istanza Vulkan nativa, restituendola all'API C++ sottoforma di puntatore opaco.

```
#[no_mangle]
pub unsafe extern "C" fn function(...) -> *mut c_void {}
```

Listing 6: Esempio di FFI in Rust

Pointer Punning per l'estrazione di memoria

Il problema più critico si presenta quando si deve gestire la *Swapchain*. OpenXR richiede che i fotogrammi vengano renderizzati direttamente sulle sue `VkImage`. Tuttavia, le strutture interne di WebGPU possiedono campi strettamente privati, impedendo la costruzione o la manipolazione di texture a partire da memoria pre-allocata esternamente.

La soluzione sfrutta una tecnica di accesso diretto alla memoria, nota come *Pointer Punning*. Sapendo per costruzione che il primo campo della struttura `Texture` nel backend Vulkan di WebGPU corrisponde all'handle nativo `ash::vk::Image`, è possibile bypassare la protezione del compilatore castando direttamente l'indirizzo di memoria della struttura ad un puntatore Vulkan.

```
#[no_mangle]
pub unsafe extern "C" fn wgpuTextureGetVulkanImage(
    texture: crate::native::WGPUTexture,
) -> *mut c_void {
    #[cfg(all(
        any(target_os="windows", target_os="linux"),
        feature="vulkan"
    ))]
    {
        let texture = texture.as_ref().expect();
        let hal_tex =
            texture.context.texture_as_hal::<hal::api::Vulkan>(texture.id);
        if let Some(hal_device) = hal_device {
            return &hal_device.raw_device().handle() as *const _ as *mut c_void;
        }
        std::ptr::null_mut()
    } // Fallback per altri OS
}
```

Listing 7: Estrazione di una `VkImage`

Giustificazione architetturale

Inizialmente si era valutata l'ipotesi di istanziare un oggetto `WGPUTexture` direttamente sopra la memoria allocata da OpenXR, tecnica detta *Memory Aliasing*. Questo approccio è stato tuttavia scartato, in favore dell'estrazione diretta della `VkImage` descritta nel Listing 7, per due ragioni fondamentali legate alla sicurezza del ciclo di vita della memoria, il quale è fondamentale per il corretto funzionamento soprattutto in una codebase Rust.

1. Se le due API condividessero lo stesso blocco di memoria, la distruzione al termine del programma genererebbe conflitti fatali, poiché entrambi i sistemi tenterebbero di deallocare la stessa risorsa. Questa situazione è anche nota come *Double-Free*.
2. L'architettura implementata separa le responsabilità; WebGPU renderizza sulle proprie textures, mentre OpenXR alloca la sua Swapchain. Sfruttando la funzione nel Listing 7, il motore C++ può eseguire una copia diretta dei pixel ad un bassissimo livello di Vulkan; quest'ultima tecnica è detta *Command Copying* o *Blitting*, e si può eseguire tramite la funzione `vkCmdCopyImage`.

Estensioni Vulkan

L'integrazione richiede l'accesso alle interfacce FFI per bypassare l'astrazione di WebGPU ed estrarre le istanze `VkInstance`, `VkDevice` e `VkPhysicalDevice`. Sebbene l'estrazione dei puntatori vada a buon fine, il test evidenzia un blocco strutturale al momento dell'handshake con il runtime OpenXR:

- OpenXR richiede l'attivazione di estensioni Vulkan specifiche per la condivisione della memoria, come `VK_KHR_external_memory_win32`, per permettere al compositore esterno di accedere ai framebuffer generati dall'applicazione.
- WebGPU, interrogando la scheda video, rileva il supporto di tali estensioni ma, per design di sicurezza, non le include nella lista delle estensioni abilitate durante la creazione del `VkDevice` logico.
- Di conseguenza, quando OpenXR tenta di invocare le funzioni di memoria condivisa, si verifica un *Segmentation Fault* a livello di driver, rendendo impossibile il rendering.

L'unico metodo per aggirare questo limite consiste nella modifica diretta e ricompilazione del codice sorgente dell'implementazione di WebGPU, nel sottomodulo `wgpu-hal`², iniettando forzatamente le estensioni richieste.

4.2.2 Estensioni di Vulkan

Riprendendo il flusso di lavoro descritto nel punto precedente, l'aggiunta delle estensioni richiede la modifica di ulteriore codice sorgente di WebGPU, in questo caso direttamente nella repository dell'API.

Sebbene solitamente l'attivazione di estensioni nel contesto di Vulkan sia complicato e tedioso, sembra che il W3C metta a disposizione un meccanismo di abilitazione di queste molto semplice da manipolare, anche se non è accessibile dall'API direttamente; esiste infatti una funzione, all'interno del file sorgente `adapter.rs`³, che già si occupa di richiedere ed attivare le estensioni di Vulkan che sono necessarie per il corretto funzionamento di WebGPU e si possono aggiungere quelle necessarie all'inizio di questa (vedi listing 8).

Queste aggiunte sono state svolte solamente nel sorgente menzionato, in quanto entrambe `VK_KHR_external_memory` e `VK_KHR_external_memory_win32` sono estensioni di tipo *device*.

²<https://github.com/gfx-rs/wgpu/tree/trunk/wgpu-hal>

³<https://github.com/lucamazza/wgpu/blob/v29h/wgpu-hal/src/vulkan/adapter.rs>

```

fn get_required_extensions(&self, requested_features: wgt::Features) -> Vec<&'static CStr> {
    let mut extensions = Vec::new();

    // Note that quite a few extensions depend on the `VK_KHR_get_physical_device_properties`
    // We enable `VK_KHR_get_physical_device_properties2` unconditionally (if available).

    // Add OpenXR required extensions to device
    extensions.push(khr::external_memory::NAME);
    extensions.push(khr::external_memory_win32::NAME);

    // Require `VK_KHR_swapchain`
    extensions.push(khr::swapchain::NAME);

    if self.device_api_version < vk::API_VERSION_1_1 {
        //... rest of the function ...
    }
}

```

Listing 8: Abilitazione estensioni di Vulkan

4.3 Transizione a DirectX12

Durante la fase di sperimentazione empirica dell'integrazione tra WebGPU e OpenXR sul backend Vulkan, sono emerse criticità bloccanti legate sia all'ecosistema dei driver che alle estensioni di memoria condivisa. Di seguito vengono analizzate le motivazioni che hanno imposto il passaggio al backend Direct3D 12 (D3D12) e le soluzioni tecniche adottate.

4.3.1 Criticità del Backend Vulkan

L'adozione iniziale di Vulkan come minimo comune denominatore si è scontrata con due problemi fondamentali:

1. Sulle macchine di test dotate di architettura grafica ibrida o GPU integrate Intel, l'allocazione e la condivisione inter-processo delle `VkImage` tramite l'estensione che serve per il funzionamento, `VK_KHR_external_memory_win32` ha causato crash imprevedibili a livello di driver grafico.
2. Anche eseguendo l'applicazione di riferimento ufficiale Khronos, `helloxr`, configurata per utilizzare il backend Vulkan, il runtime OpenXR (es. SteamVR/Oculus) non è stato in grado di completare la creazione della sessione o la presentazione della swapchain sull'ambiente di sviluppo.

Poiché l'ambiente target primario è il sistema operativo Windows, si è scelto di effettuare un pivot architetturale verso DirectX12. D3D12 rappresenta l'API nativa d'elezione per i runtime OpenXR su Windows, garantendo stabilità nativa e bypassando le complicazioni legate all'esportazione manuale dei semafori e della memoria condivisibile di Vulkan.

4.3.2 Estrazione Handle D3D12 e ownership in wgpu-native

Per consentire l'interoperabilità tra wgpu-native e OpenXR, sono stati estesi i sorgenti Rust della libreria esponendo funzioni FFI dedicate all'estrazione degli oggetti COM sottostanti: `IDXGIFactory`, `IDXGIAdapter`, `ID3D12Device` e `ID3D12CommandQueue`.

Un errore critico iniziale durante la chiamata ad `xrCreateSession` provocava un'eccezione di violazione d'accesso nel loader OpenXR (*invalid jump*).

In ambiente Windows, gli oggetti dell'API Direct3D12 (come ID3D12Device e ID3D12Image) sono oggetti COM (*Component Object Model*). La loro memoria viene gestita attraverso il meccanismo del Reference Counting (conteggio dei riferimenti), regolato dai metodi dell'interfaccia IUnknown: AddRef() incrementa il contatore, mentre Release() lo decrementa. Quando il contatore raggiunge lo zero, l'oggetto viene deallocato autonomamente dalla memoria video.

La violazione di accesso è dovuta proprio a questa necessità di contare i riferimenti e la necessità di introdurre la clonazione esplicita dell'interfaccia in Rust (iface.clone()) prima di esportare il puntatore grezzo deriva dal conflitto tra la gestione dell'ownership in C++ tramite Microsoft::WRL::ComPtr e quella del backend Rust di wgpu-native.

```
unsafe fn into_raw_ptr_addrf<T: Interface>(iface: &T) -> *mut c_void {
    let cloned = iface.clone();
    cloned.into_raw() as *mut c_void
}
```

Listing 9: Reference Counting per il ritorno di Handle nativi

Questa funzione helper viene poi utilizzata al ritorno degli handle per estenderne il lifetime e non lasciare che vengano distrutti prima che il codice di destinazione possa anche farne uso. Nel seguente listing 10 il getter per un esempio di handle nativo D3D12 con utilizzo della funzione sovraccitata.

```
#[no_mangle]
pub unsafe extern "C" fn wgpuTextureGetD3D12Image(
    texture: native::WGPUTexture
) -> *mut c_void {
    #[cfg(all(any(target_os = "windows"), feature = "dx12"))]
    {
        let texture = texture.as_ref().expect("invalid texture");
        let hal_texture =
            texture.context.texture_as_hal::<hal::api::Dx12>(texture.id);
        if let Some(hal_texture) = hal_texture {
            let res = hal_texture.raw_resource();
            return into_raw_ptr_addrf(res);
        }
        std::ptr::null_mut()
    }
    #[cfg(not(all(any(target_os = "windows"), feature = "dx12")))]
    {
        std::ptr::null_mut()
    }
}
```

Listing 10: Funzione di recupero degli handle D3D12

Il reference counting è anche implementato nella parte C++, nella quale ogni handle estratto viene salvato all'interno di un microsoft::wrl::ComPtr, mantenendo l'assegnazione corretta *non-owning* dei puntatori a OpenXR.

```
m_dx12->graphicsBinding = {XR_TYPE_GRAPHICS_BINDING_D3D12_KHR};
m_dx12->graphicsBinding.device = m_dx12->d3d12Device.Get();
m_dx12->graphicsBinding.queue = m_dx12->d3d12Queue.Get();
```

Listing 11: Assegnazione *Non-Owning* di puntatori al Graphics Binding

4.3.3 Sincronizzazione e composizione

L'integrazione nativa tra l'API grafica e il runtime VR raggiunge la sua massima complessità all'interno del ciclo applicativo principale. Diversamente da approcci ad alto livello, l'utilizzo di una *Render Hardware Interface* esplicita come WebGPU richiede un'orchestrazione manuale di ogni singola fase della pipeline, dalla sincronizzazione temporale al trasferimento dei dati in memoria video.

Ciclo di vita del frame

OpenXR impone un rigoroso *frame pacing* per garantire che le immagini vengano presentate all'Head Mounted Display minimizzando la latenza *motion-to-photon*. Il ciclo di vita di un singolo fotogramma stereoscopico si articola attraverso tre chiamate fondamentali dettate dal runtime:

1. `xrWaitFrame`: Sincronizza il thread dell'applicazione con le tempistiche del compositor, mettendo il processo in attesa fino al momento ottimale per iniziare l'elaborazione del frame successivo.
2. `xrBeginFrame`: Segnala formalmente al runtime l'inizio della composizione grafica.
3. `xrEndFrame`: Conclude il fotogramma, sottomettendo un array di livelli di composizione (`XrCompositionLayerBaseHeader`) al compositor per la visualizzazione finale.

Nei motori grafici didattici tradizionali, il ciclo applicativo esegue il rendering in modo continuo e incondizionato finché la finestra è attiva. Al contrario, OpenXR implementa una macchina a stati complessa in cui è il runtime VR (es. SteamVR) a dettare le tempistiche e la necessità effettiva di generare un nuovo fotogramma, al fine di garantire un'esperienza priva di *stuttering* e minimizzare la latenza.

Come documentato in implementazioni precedenti basate su OpenVR, l'utente aveva la responsabilità di creare i framebuffer e inviarli al runtime. OpenXR inverte questo paradigma: l'applicazione deve richiedere le immagini e sottostare allo stato della sessione. Durante l'esecuzione, lo stato della sessione può transitare da `XR_SESSION_STATE_FOCUSED` a `XR_SESSION_STATE_VISIBLE` (ad esempio, quando l'utente apre la dashboard di sistema del visore).

In questi stati intermedi, la struttura `XrFrameState` restituita da `xrWaitFrame` imposta il flag `shouldRender` a `XR_FALSE`. Ignorare questo flag o interrompere la catena di chiamate (`xrWaitFrame` → `xrBeginFrame` → `xrEndFrame`) corrompe la sincronizzazione della swap-chain, portando a crash per violazione di accesso alla memoria video. L'integrazione corretta, implementata nella classe `XrWGPUBridge`, richiede di bypassare la logica di WebGPU ma di invocare comunque `xrEndFrame` sottomettendo zero *composition layers*, mantenendo intatto il *frame pacing* senza sprecare cicli di calcolo.

Acquisizione pose

Una volta iniziato il frame, è necessario interrogare il sistema di tracciamento spaziale per ottenere la posizione e la rotazione precise dell'utente. Questo avviene tramite la chiamata `xrLocateViews`, che popola un array di strutture `XrView`, contenenti il campo visivo (FOV) e la posa per ogni occhio.

L'infrastruttura sviluppata richiede la costruzione manuale delle matrici di trasformazione. I dati di tracking estratti (espressi come quaternioni per la rotazione e vettori per la traslazione) vengono convertiti e combinati per generare la matrice della telecamera. Per posizionare correttamente la geometria nel View Space di WebGPU, viene calcolata la matrice di Vista inversa ($M_{view} = M_{camera}^{-1}$), garantendo che la scena reagisca coerentemente ai movimenti fisici dell'HMD.

```

MatricesUniforms matrices = {};
glm::quat eyeOrientation(
    currentView.pose.orientation.w,
    currentView.pose.orientation.x,
    currentView.pose.orientation.y,
    currentView.pose.orientation.z
);
glm::vec3 eyePosition(
    currentView.pose.position.x,
    currentView.pose.position.y,
    currentView.pose.position.z
);
glm::mat4 rotation_matrix = glm::mat4_cast(eyeOrientation);
glm::mat4 translation_matrix = glm::translate(glm::mat4(1.0f), eyePosition);
glm::mat4 eye_transform = translation_matrix * rotation_matrix;
glm::mat4 view_matrix = glm::inverse(eye_transform);
glm::mat4 model_matrix = glm::mat4(1.0f);

```

Listing 12: Costruzione dinamica della matrice di View tramite dati OpenXR

Resource Barriers

L'utilizzo diretto di comandi hardware come CopyResource introduce una severa complessità legata alla rigida gestione degli stati di memoria imposta dalle API moderne come Direct3D 12 e Vulkan.

WebGPU, al termine della codifica del suo *Render Pass*, lascia la propria texture in uno stato designato alla scrittura (D3D12_RESOURCE_STATE_RENDER_TARGET). Al tempo stesso, OpenXR fornisce le immagini della sua swapchain preconfigurate in uno stato simile, attendendosi che l'applicazione le restituisca intatte.

Affinché la copia a basso livello vada a buon fine senza causare un fallimento silenzioso della command queue (che si tradurrebbe in un output visivo nel visore completamente nero, nonostante la corretta sincronizzazione del loop e l'assenza di errori di compilazione), è imperativo inserire delle *Resource Barrier*. Queste barriere indicano al driver di transire esplicitamente la texture sorgente e quella di destinazione negli stati COPY_SOURCE e COPY_DEST, per poi ripristinarle in sicurezza a copia ultimata.

```

D3D12_RESOURCE_BARRIER preBarriers[2] = {};
preBarriers[0].Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
preBarriers[0].Transition.pResource = srcWebGPU;
preBarriers[0].Transition.StateBefore = D3D12_RESOURCE_STATE_RENDER_TARGET;
preBarriers[0].Transition.StateAfter = D3D12_RESOURCE_STATE_COPY_SOURCE;
preBarriers[0].Transition.Subresource = D3D12_RESOURCE_BARRIER_ALL_SUBRESOURCES;
preBarriers[1].Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
preBarriers[1].Transition.pResource = dstOpenXR;
preBarriers[1].Transition.StateBefore = D3D12_RESOURCE_STATE_RENDER_TARGET;
preBarriers[1].Transition.StateAfter = D3D12_RESOURCE_STATE_COPY_DEST;
preBarriers[1].Transition.Subresource = D3D12_RESOURCE_BARRIER_ALL_SUBRESOURCES;
m_dx12->commandList->ResourceBarrier(2, preBarriers);
m_dx12->commandList->CopyResource(dstOpenXR, srcWebGPU);

```

Listing 13: Gestione degli stati di memoria tramite D3D12 Resource Barriers

Gestione della Swapchain e Rendering Offscreen

Come analizzato precedentemente, l'assenza di primitive *zero-copy* stabili per il wrapping diretto delle immagini di OpenXR in WebGPU ha imposto un approccio indiretto. L'architettura sviluppata all'interno della classe `XrWGPUBridge` delega a OpenXR la creazione della swapchain hardware nativa, ma alloca parallelamente una `WGPUTexture offscreen` dedicata per ciascuna *view* (occhio sinistro e occhio destro).

Durante l'inizializzazione, per ogni immagine richiesta dal compositor, viene istanziata una texture WebGPU configurata esplicitamente con i flag `WGPUTextureUsage_RenderAttachment` e `WGPUTextureUsage_CopySrc`. Quest'ultimo flag è cruciale, in quanto permette al backend di autorizzare operazioni di copia a basso livello a valle del processo di rendering. Il puntatore D3D12 nativo di questa texture offscreen viene estratto e salvato nel contesto della swapchain per l'operazione finale di trasferimento.

4.4 Altri tentativi

Durante lo sviluppo della demo, sono stati svolti altri tentativi che purtroppo non hanno portato ad alcun risultato, ma sono comunque meritevoli di nota, sia per documentare il lavoro svolto, sia per virarne alla larga in un eventuale ripresa del presente lavoro.

4.4.1 Passaggio a Dawn

Con scopi di ricerca, per dimostrare anche per quali motivi le vie alternative non siano percorribili, per evitare lavoro extra in futuro, si è provato a percorrere tali strade alternative; una di queste riguarda l'utilizzo di **Dawn**, implementazione in C++ di WebGPU sviluppata da Google ed utilizzata come backend per il browser *Chromium*. Architetturealmente, Dawn presenta una scelta più promettente: essendo sviluppata nativamente in C++, offre una superficie API teoricamente più flessibile rispetto ai rigidi binding C basati su Rust, offerti da `wgpu-native`.

Il requisito fondamentale per l'integrazione con un runtime VR tramite OpenXR è la condivisione del contesto grafico. OpenXR opera ad un livello molto vicino all'hardware; per poter comporre l'immagine stereoscopica da inviare al visore, il runtime VR deve conoscere esattamente quale istanza e quale device Vulkan (oppure DirectX/OpenGL) stanno generando i frame. Nello specifico, per un backend Vulkan, OpenXR richiede l'accesso esplicito agli handle nativi `VkInstance`, `VkPhysicalDevice` e `VkDevice`.

L'approccio iniziale prevede di mantenere WebGPU come driver dell'applicazione; Dawn deve inizializzare il contesto grafico, creare il device logico e, successivamente, l'applicazione avrebbe dovuto estrarre questi handle nativi da passare ad OpenXR.

Analisi della superficie API Dawn

L'analisi della codebase di Dawn rivela che, sebbene in passato esistessero funzioni per l'interoperabilità nativa, proprio come per `wgpu-native`, queste sono soggette a deprecazione; il design dell'API di Google fortemente orientato all'incapsulamento, nascondendo deliberatamente tutti i dettagli sottostanti alle astrazioni.

Per poter fornire ad OpenXR i dati necessari, è necessario tentare di estrarre questi dai dati encapsulati nei tipi definiti da Dawn; essendo assenti tali funzioni si rende di nuovo necessaria l'estensione manuale dell'API con funzioni personalizzate. Queste modifiche vedono due obiettivi:

1. L'implementazione di funzioni "getter" personalizzate, all'interno del namespace dedicato `dawn::native::vulkan` per esporre i puntatori agli oggetti `VkInstance` e `VkDevice` nascosti all'interno dei relativi oggetti `Adapter` e `Device` di Dawn.

2. OpenXR non si limita a richiedere il device, ma impone che quest'ultimo venga creato con specifiche estensioni Vulkan abilitate, come `VK_KHR_external_memory` oppure altre estensioni per il timing del visore. Dawn normalmente alloca il device richiedendo solamente le estensioni strettamente necessarie per il funzionamento base di WebGPU.

Procedendo con un tentativo di modifica diretta, tramite forking e patching del codice sorgente di Dawn, ciò richiede l'alterazione del processo di inizializzazione all'interno del file `dawn/native/vulkan/DeviceVk.cpp`⁴, per forzare l'inclusione delle estensioni richieste.

Conflitto filosofico con lo standard W3C

Durante l'implementazione di tali modifiche sono emerse criticità architetturali insormontabili, non legate a bug o imperfezioni del codice, bensì alla filosofia stessa con cui WebGPU è concepita dal W3C.

L'API infatti è progettata per essere "Sandboxed", ovvero *confinata*. Il suo scopo primario è di eseguire carichi di lavoro grafici e calcolo in modo sicuro all'interno di un web browser, prevenendo per design l'accesso arbitrario a qualsiasi settore della memoria GPU o al sistema operativo host. Esporre gli handle di Vulkan nudi e crudi oppure permettere l'iniezione di estensioni esterne non verificate da WebGPU viola questo principio proprio alla base, e risulta quindi molto difficoltoso ed impossibile da mantenere in sicurezza.

Le criticità si riassumono quindi in tre punti principali:

- Anche riuscendo ad estrarre il Device, OpenXR richiede di renderizzare direttamente sulle immagini della propria Swapchain, ottenibili tramite `xrEnumerateSwapchainImagesKHR`. Dawn non fornisce alcun metodo standardizzato e stabile per wrappare un array di `VkImage` esterne, in oggetti di tipo `WGPUTexture`, destinati al rendering continuo, senza dover passare per costose copie di memoria.
- Modificare il backend di Dawn ha richiesto l'uso di hack invasivi. L'API è in continua e rapida evoluzione, con commit giornalieri e features non ancora completamente definite per quanto riguarda l'architettura. Mantenere una versione "patched" di questa per supportare funzionalità VR si traduce in una condanna all'obsolescenza immediata e periodica. Inoltre, sempre per le stesse motivazioni di sicurezza, il W3C non sembra aver intenzione di supportare estensioni o plugin per estendere le funzionalità dell'API.
- Le discussioni sui repository ufficiali del W3C e di Dawn confermano che non vi è alcuna intenzione, almeno nel breve termine, di standardizzare l'esportazione di handle nativi. Qualsiasi *Pull Request* volta ad aggiungere funzioni verrebbe respinta in quanto contrasterebbe la filosofia corrente.

```
dawn::native::vulkan::ExternalImageExportInfoVk exportInfo;
//          ^ Non è possibile istanziare oggetti di questo tipo
bool ok = dawn::native::vulkan::ExportVulkanImage(
    m_eyeTargets[i].wgpuOffscreenTexture.Get(),
    VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
    &exportInfo
);
```

Listing 14: Costruttore inaccessibile per `ExternalImageExportInfoVk`

⁴<https://github.com/google/dawn/blob/main/src/dawn/native/vulkan/DeviceVk.h>

Out-of-Scope

Alla luce dei risultati ottenuti, è apparso chiaro che proseguire su questa strada avrebbe significato deviare completamente dagli obiettivi del presente progetto. Il mandato richiede uno studio di fattibilità dell'interoperabilità tra WebGPU e i runtime VR, non lo sviluppo ed il mantenimento di un backend grafico custom.

Costringere l'API a comportarsi in un modo che ricade al di fuori della filosofia architetturale e di sviluppo per la quale è pensata, modificandone il codice sorgente, genera un notevole debito tecnico, specialmente in contesto didattico dove gli studenti devono potersi concentrare sulla logica VR, e non il debugging di un'API frammentata ed instabile.

In conclusione, il tentativo di utilizzare WebGPU come entità creatrice delle risorse native e modificarne il core per estrarle è formalmente abbandonato, in quanto le risorse necessarie per tale estrazione rendono il risultato inutilizzabile per il corso di realtà virtuale.

4.4.2 Inversion of Control (IoC)

Il fallimento descritto negli atti finali della sezione precedente evidenzia un'alternativa via percorribile per aprire il dialogo tra questi due mondi chiusi. Se risulta infattibile l'esposizione di risorse native create da WebGPU verso OpenXR, è necessario ribaltare il paradigma ed applicare un pattern di inversione di controllo. Ciò consiste nella creazione delle risorse native al di fuori del contesto di WebGPU, direttamente tramite istruzioni Vulkan native, per poi crearvi attorno gli oggetti di contesto dell'API WebGPU.

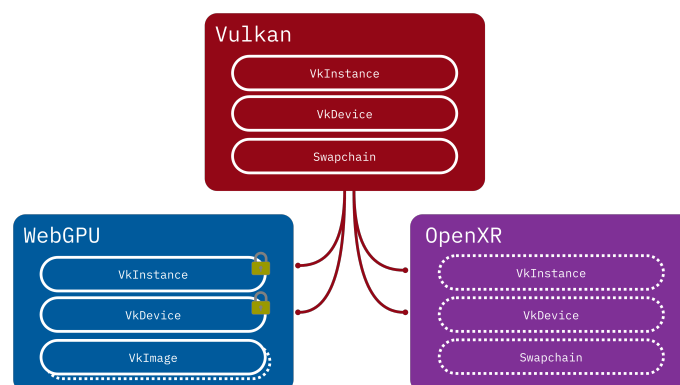


Figura 4.4: Approccio *Inversion of Control*

Limiti di esportazione memoria

L'indagine volta all'utilizzo di WebGPU come entità principale per la creazione del contesto grafico si è scontrata con limitazioni architetturelle intrinseche al design dell'API. Come analizzato precedentemente, l'integrazione nativa con OpenXR richiede una rigorosa condivisione delle risorse Vulkan⁵, e soprattutto, una sincronizzazione inter-process per la gestione delle *swapchain images*.

L'analisi dei sorgenti di Dawn ha rivelato che le funzioni sperimentali di interoperabilità, come `ExportVulkanImage` sono soggette a forti instabilità e mancano di simmetrie tra le piattaforme backend. In generale si nota che piattaforme supportate da molteplici sistemi operativi, come Vulkan, sono lasciate in secondo piano e viene rimossa la possibilità di interoperare direttamente, in favore di piattaforme native che sono usualmente il "default" per il sistema operativo (Metal per macOS, DirectX per Windows e *File Descriptors* per Linux). Nello specifico, si è

⁵Vulkan è l'unico backend utilizzato da WebGPU che è direttamente compatibile con almeno 2 dei 3 sistemi operativi maggiormente utilizzati

constatato che tipi di dato, come per esempio l'enum `ExternalImageType`, necessario per la dichiarazione dei tipi di handle del sistema operativo da esportare per i *semaphores* di Vulkan, supporta nativamente i descrittori per *Android*, *macOS*, *Linux*, ma non ha alcun supporto per gli handle nativi di Windows (vedi Listing 15).

```
// The different types of external images
enum ExternalImageType : uint16_t {
    OpaqueFD,           // Linux
    DmaBuf,             // ChromeOS
    IOSurface,          // Metal
    EGLImage,           // Android
    GLTexture,          // OpenGL
    AHardwareBuffer,    // Android
    Last = AHardwareBuffer,
};
```

Listing 15: Enumerativo `ExternalImageType`

Questa mancanza rende di fatto impossibile, allo stato attuale dell'arte, implementare una sincronizzazione *zero copy* tra WebGPU e OpenXR in ambiente Windows, senza ricorrere a pesanti alterazioni del codice sorgente delle librerie grafiche.

Questa mancanza oltretutto non è una svista, ma deriva dalle priorità di sviluppo del backend di Google: su Windows, Dawn predilige l'interoperabilità tramite `DirectX`, mentre riserva l'integrazione nativa Vulkan (e la relativa esportazione di *File Descriptors*) principalmente agli ambienti Linux e Android. Di conseguenza, il ponte Vulkan-Windows risulta essere un caso d'uso non ancora supportato, bloccando di fatto la condivisione Zero-Copy.

Si nota che è molto evidente l'intenzione dei designer e sviluppatori dell'API di WebGPU di proibire o rendere molto complesso ciò che si sta provando a compiere. È evidente che le limitazioni vengono direttamente dalla specifica e dalla filosofia di WebGPU.

Valutazione del debito tecnico

Per superare queste limitazioni si è valutata la possibilità di effettuare un fork del repository e alterarne il codice sorgente per forzare l'iniezione delle estensioni Vulkan richieste da OpenXR ed esporre deliberatamente i puntatori nudi, aggirando l'incapsulamento.

Come già menzionato nelle sezioni precedenti, tale approccio viola apertamente i principi fondamentali definiti dal W3C per WebGPU, la quale nasce come *API Sandboxed* orientata alla sicurezza ed all'isolamento. Ciò rende inutile anche pensare di aggirare il corrente problema con una modifica all'API per introdurre i tipi mancanti.

Zero-Copy vs Workaround via RAM

Alla luce della documentata assenza delle primitive IPC per l'ambiente Windows, è emerso che qualsiasi approccio nativo sulla memoria GPU tra Dawn e OpenXR è precluso. L'unico approccio tecnicamente percorribile per estrarre il fotogramma generato da WebGPU e trasmetterlo all'HMD consiste nell'aggirare la memoria video, forzando un trasferimento dei pixel attraverso la memoria di sistema. Questa soluzione di ripiego modifica l'architettura del sistema come segue:

- WebGPU esegue il rendering stereoscopico della scena all'interno delle proprie Texture isolate.

- Terminata la renderizzazione, i dati dei pixel vengono copiati dalle texture della GPU ad un oggetto Buffer.
- Tramite la funzione MapAsync, il buffer viene mappato nella memoria della CPU. Questo passaggio forza la sincronizzazione tra CPU e GPU, consentendo all'applicazione di leggere l'array di byte crudi dei pixel in formato *RGBA*.
- I dati, ora residenti nella RAM, vengono presi in carico dai comandi Vulkan nativi, indipendenti da WebGPU. Tramite uno staging buffer, questo viene caricato in memoria GPU come *VkImage*.

Sebbene il passaggio del fotogramma tramite RAM permetta, a livello concettuale, di completare la pipeline visiva ottenendo un Proof of Concept funzionante, questa architettura è fallimentare per applicazioni VR.

Il trasferimento continuo di fotogrammi ad altissima risoluzione (per esempio 2000x2000 per occhio, a 90 o 120 FPS) lungo il bus PCIe dalla GPU alla CPU, e successivamente di nuovo dalla CPU alla GPU, genera un bottleneck gravissimo. Questo processo introduce una massiccia *Motion-to-Photon latency*, ovvero un ritardo critico tra il movimento della testa dell'utente e l'aggiornamento dell'immagine nel visore. Nella Realtà Virtuale moderna, questa latenza deve essere categoricamente minimizzata per prevenire il rischio di motion sickness.

4.5 Conclusione

Lo sviluppo del prototipo ha dimostrato con successo la fattibilità tecnica dell'integrazione tra WebGPU e OpenXR in ambiente nativo. Nonostante l'API del W3C nasca con una forte vocazione alla sicurezza ed al sandboxing, caratteristiche che ne complicano l'interfacciamento diretto con la memoria di altri processi, come evidenziato nelle analisi dell'API; l'utilizzo mirato dell'infrastruttura sottostante ha permesso di colmare il divario architetturale.

Rispetto alle architetture didattiche precedentemente impiegate nel corso, basate sull'accoppiata OpenGL e OpenVR, il nuovo motore grafico impone un cambio di paradigma. Si abbandona il tradizionale stile di programmazione unico ad OpenGL, per passare ad uno più conforme al moderno Vulkan, come d'altronde da intenzione dei creatori. L'overhead di questo cambiamento è però relativo, in quanto anche se le risorse da istanziare sono molteplici e va ricordata ogni loro specifica, lo stile di creazione di ognuna è praticamente lo stesso, rendendo la stesura del codice riassumibile tramite una checklist.

Nonostante l'aggiuntivo livello di API aggiunto da WebGPU e la traduzione in API native, il paradigma da questo adottato fornisce comunque abbondanti livelli di controllo sull'hardware grafico.

In conclusione, l'architettura implementata soddisfa integralmente i requisiti ingegneristici del mandato, in quanto gestisce correttamente l'allocazione e la copia asincrona sulle swap-chain stereoscopiche, estrae dinamicamente le pose dal sistema di tracciamento per costruire le matrici di vista asimmetriche, rispetta la rigorosa macchina a stati ed il frame pacing imposti dal runtime OpenXR.

Avendo stabilito e validato un'infrastruttura di rendering solida e funzionante, il prototipo fornisce la base ideale per procedere nel prossimo capitolo, con la fase di profilazione. In questa fase infatti verranno analizzate le prestazioni del sistema, misurando oggettivamente le latenze ed i framerate per determinare l'importanza dell'eventuale overhead e quanto le performance finali lo giustifichino.

Capitolo 5

Test e validazione

Questo capitolo presenta l'analisi empirica e la valutazione delle prestazioni dell'architettura WebGPU-OpenXR sviluppata nel corso del progetto. L'obiettivo primario di questa fase è verificare se l'elevata complessità di sviluppo e la rigorosa gestione esplicita delle risorse introdotta dalla nuova API portino a vantaggi misurabili o se, al contrario, evidenzino colli di bottiglia architetturali.

Per condurre questa analisi, il prototipo basato su WebGPU verrà confrontato direttamente con le soluzioni tradizionali precedentemente impiegate nei corsi accademici della SUPSI, nello specifico quelle basate sull'integrazione tra OpenGL e OpenVR. Il confronto si articolerà su due direttrici fondamentali:

- Verrà eseguita un'analisi critica della complessità intrinseca delle API, valutando la verbosità del codice, la difficoltà nella gestione della memoria e la curva di apprendimento. Questi fattori sono determinanti per stimare la reale sostenibilità didattica dell'adozione di WebGPU in un corso bachelor.
- Verranno effettuate misurazioni oggettive focalizzandosi sui due parametri più critici nel dominio della Realtà Virtuale: il frame rate (FPS) e la latenza *motion-to-photon*. Garantire prestazioni ottimali in questi due ambiti è essenziale per mantenere l'immersività e scongiurare fenomeni di *motion sickness*.

I dati e le osservazioni raccolte in questo capitolo forniranno la base empirica necessaria per formulare, nelle conclusioni, un verdetto definitivo sulla fattibilità e l'adeguatezza di WebGPU come standard moderno per l'insegnamento della Realtà Virtuale.

È anche da decidere, in che modo verrà poi sviluppato il motore da parte dello studente, quanto del codice ponte sia di sua responsabilità e quanto gli venga consegnato come blackbox.

5.1 Complessità e Overhead di Sviluppo

Il mandato del presente progetto richiede un confronto oggettivo tra l'approccio WebGPU e le soluzioni tradizionali in termini di complessità di sviluppo.

La transizione da un'API implicita come OpenGL 1.1/2.0 a una *Render Hardware Interface* esplicita come WebGPU introduce un debito tecnico iniziale significativo. Studi precedenti condotti sull'adozione di WebGPU per la computer grafica didattica hanno stimato che, per renderizzare una scena di base, il rapporto tra codice applicativo e codice di configurazione (*boilerplate*) subisce un'inversione drastica. Se in OpenGL la logica applicativa copre circa l'80% del codice e il setup il 20%, in WebGPU il setup delle risorse (buffer, layout, pipeline) occupa fino al 70% del codice, lasciando solo il 30% alla logica.

Questa complessità è ulteriormente amplificata nel dominio VR. L'infrastruttura sviluppata in questo progetto dimostra che:

1. A differenza di librerie preesistenti come OvVR, che nascondono i dettagli di basso livello agli studenti, WebGPU obbliga lo sviluppatore a comprendere concetti avanzati come l'allineamento della memoria a 16 byte per le `struct` in C++ e le rigorose regole di validazione dei *Bind Group*.
2. L'utilizzo di binding C++ derivati da implementazioni Rust (`wgpu-native`) costringe la risoluzione manuale di librerie di sistema Windows a livello di linker (`Ws2_32`, `Bcrypt`, ecc.), aggiungendo frizione nel setup dell'ambiente di sviluppo nativo.
3. WebGPU si basa su un sistema rigoroso di validazione. Sbagliare un allineamento di memoria o un flag di utilizzo (*Usage Flag*) si traduce in log di errore granulari o, in assenza di una gestione asincrona degli errori (*Uncaptured Error Callback*), nel fallimento silenzioso del render pass.

Tuttavia, questo sforzo ingegneristico restituisce un controllo assoluto sull'hardware, allineando le competenze acquisite dagli studenti con l'architettura delle moderne API grafiche industriali, giustificando la complessità aggiuntiva per corsi di livello avanzato.

5.2 Performance

La seconda analisi valuta l'adeguatezza del sistema per l'utilizzo in Realtà Virtuale. Lo sviluppo VR impone tolleranze minime: il Framerate (FPS) deve rimanere stabilmente ancorato alla frequenza di aggiornamento del visore e la latenza *motion-to-photon* (il tempo che intercorre dal movimento della testa all'aggiornamento dei pixel) deve rimanere al di sotto dei 20 millisecondi per prevenire fenomeni di *motion sickness*. Anche se un requisito generale dei visori è la stabilità a 90 FPS, i visori simulati da telefono cellulare, utilizzati durante questo progetto come il corso di realtà virtuale, sono spesso limitati a 60 FPS.

I test sono stati condotti eseguendo la medesima scena 3D stereoscopica sia sull'infrastruttura OpenGL/OvVR sia sul prototipo WebGPU/OpenXR.

5.2.1 VRidge + iPhone

Questa serie di performance test è eseguita con il software *VRidge* come supporto; questo permette di eseguire ed interagire con applicazioni di realtà virtuale, utilizzando il proprio telefono cellulare come HMD, utilizzando *SteamVR* come runtime VR. Si noti che questo supporto introduce notevoli bottleneck, dovuti alla connessione poco stabile con i cellulari e la bassa capacità di refresh rate degli stessi. Queste performance hanno comunque validità per il corso, in quanto è il supporto principale utilizzato dallo studente, quindi il previsto utilizzo dei progetti di corso.

Framerate (FPS)

Il Framerate (FPS), misurato in appunto fotogrammi per secondo, rappresenta la metrica prestazionale più immediata per valutare la fluidità di un'applicazione grafica. In realtà virtuale, mantenere un framerate elevato e costante è un requisito tecnico critico per garantire l'immersività e prevenire fenomeni di motion sickness.

Questa sezione mette a confronto la stabilità di Framerate (FPS) tra l'implementazione nativa in OpenGL+OpenVR ed il nuovo prototipo WebGPU+OpenXR, analizzando non solo la media generale, ma anche picchi minimi e massimi oltre al 99esimo percentile, con lo scopo di evidenziare eventuali cali di prestazione anomali.

Tabella 5.1: Test FPS (VRRidge)

FPS	OpenGL+OpenVR	WebGPU+OpenXR
Medi	~ 60 FPS	~ 60 FPS
Minimi	~ 21 FPS	~ 17 FPS
Massimi	~ 60 FPS	~ 61 FPS

Questi risultati sono presentati solamente per completezza, in quanto nell'ambiente corrente di test i frame al secondo sono limitati a 60 da sistema, in quanto proiettati su schermi di cellulari i quali sono solitamente limitati a 60Hz.

Ciononostante, si può notare che la soluzione **WebGPU+OpenXR**, soprattutto durante la fase di inizializzazione, vede un calo di frame al secondo più elevato rispetto alla soluzione già presente. Entrambe le soluzioni hanno comunque praticamente gli stessi risultati in entrambi i casi.

Frame Time

Il frame time indica il tempo effettivo, misurato in millisecondi, impiegato dal sistema per elaborare e renderizzare un singolo fotogramma. A differenza del framerate, che rende nota una media al secondo, questo fornisce una visione molto più granulare della stabilità del motore grafico. In un'applicazione OpenXR, i picchi improvvisi di questa metrica possono causare la perdita della finestra di sincronizzazione con il *compositor* del visore.

La seguente tabella permette di concludere e valutare se l'architettura esplicita di WebGPU garantisce tempi di esecuzione più costanti rispetto alle euristiche dei driver OpenGL.

Tabella 5.2: Test Frame Time (VRidge)

Frame Time	OpenGL+OpenVR	WebGPU+OpenXR
Medio	~ 14.5ms	~ 9.0ms
Minimo	~ 12.1ms	~ 8.2ms
Massimo	~ 20.1ms	~ 12.3ms

I risultati dimostrano come l'utilizzo di risorse moderne portino giovamento alle metriche di performance, soprattutto risultati dovuti probabilmente all'utilizzo delle euristiche di OpenXR. In conclusione, fatto intuibile anche senza scrivere una riga di codice, utilizzare OpenXR e WebGPU, con DirectX12 come backend offre delle performance senza eguali con l'alternativa legacy di OpenGL e OpenVR.

Utilizzo CPU

L'utilizzo del processore è un indicatore fondamentale dell'overhead introdotto dalle chiamate alle API grafiche. Uno degli obiettivi teorici delle librerie moderne come WebGPU è proprio la riduzione del carico sulla CPU, delegando la validazione dello stato alla fase di inizializzazione della pipeline grafica. Questa misurazione ha lo scopo di quantificare l'impatto sul processore durante il render loop stereoscopico, misurando soprattutto eventuali overhead computazionali di adattamento tra WebGPU e OpenXR.

Tabella 5.3: Test Utilizzo CPU (VRidge)

CPU %	OpenGL+OpenVR	WebGPU+OpenXR
Medio	~ 2.30%	~ 0.00%
Minimo	~ 1.70%	~ 0.00%
Massimo	~ 2.50%	~ 0.10%

Sorprendentemente, escludendo la fase di avvio dell'applicazione, nella quale l'utilizzo della CPU è presente anche se in minima quantità, la soluzione sviluppata durante questo progetto NON utilizza alcuna risorsa del processore per funzionare, dove invece la soluzione correntemente utilizzata nel corso ne necessita una quantità considerevole per la sua semplicità.

Utilizzo VRAM

L'utilizzo e la corretta gestione della memoria video, detta anche VRAM, è essenziale per applicazioni VR complesse, soprattutto quando fanno uso di texture ad alta risoluzione e swap-chain multiple. Come analizzato durante la fase di implementazione, WebGPU richiede un'allocazione esplicita e un rigoroso allineamento della memoria (ad esempio a 16 byte per le uniform).

Questa sezione valuta l'impronta in memoria delle due diverse architetture, per determinare se le rigide specifiche di WebGPU e il layer di interoperabilità con OpenXR comportino un consumo di VRAM significativamente maggiore rispetto all'astrazione di alto livello fornita da OpenGL.

Tabella 5.4: Test Utilizzo RAM (VRidge)

RAM %	OpenGL+OpenVR	WebGPU+OpenXR
Principale	0.9GB	0.4GB
Virtuale	0.9GB	1.3GB

È da notare come, la presenza di diverse risorse di supporto aggiuntive rispetto alla soluzione **OpenGL+OpenVR**, portano ad un utilizzo maggiore (di ben 400MB) della memoria video. Le risorse di interoperabilità tra i sistemi WebGPU e OpenXR introducono quindi ulteriore utilizzo dello spazio di memoria video, sicuramente da prendere in considerazione per applicazioni di maggiori dimensioni.

Questo è comunque un utilizzo contenuto se comparato con le VRAM moderne che hanno gran quantità di spazio. L'utilizzo della memoria principale è invece ridotto per entrambe le soluzioni, anche per

Frame "Droptati"

I fotogrammi scartati o mancanti si verificano quando il motore di rendering non riesce a consegnare l'immagine finita alla swapchain di OpenXR entro il tempo limite richiesto dal refresh rate del visore. Quando ciò accade, il runtime VR è costretto a riproiettare il fotogramma precedente, causando un visibile artefatto visivo, detto *stuttering*.

Il grafico sottostante quantifica il numero assoluto di fotogrammi scartati durante le sessioni di test di egual durata, offrendo una misura diretta della stabilità del sistema.

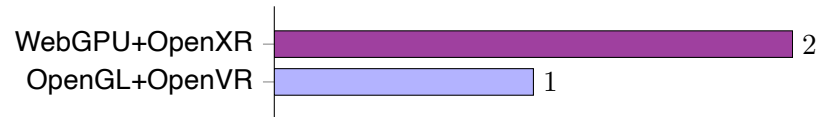


Figura 5.1: Frame "Droptati" (VRidge)

Comparativa finale

Questa sezione propone una sintesi olistica di tutte le metriche raccolte durante la profilazione tramite VRidge. Aggregando i dati relativi a framerate, tempi di esecuzione, consumo di risorse e stabilità del fotogramma, si delineerà un quadro prestazionale completo. L'obiettivo di questo confronto finale è stabilire oggettivamente se il debito tecnico e la complessità architetturale introdotti da WebGPU siano giustificati da un miglioramento misurabile delle performance, determinando così la validità tecnica della sua potenziale adozione come standard didattico.

Tabella 5.5: Comparativa finale (VRidge)

Metrica	OpenGL+OpenVR	WebGPU+OpenXR
FPS	~ 60 FPS	~ 60 FPS
Frame Time	~ 14.5ms	~ 9.0ms
CPU Usage	~ 1.70%	~ 0.00%
Uso VRAM	0.9GB	1.3GB
Frame Droptati	1	2

Capitolo 6

Risultati

6.1 Manuale d'uso

Capitolo 7

Conclusioni

7.1 Sviluppi futuri

Glossario

Chromium Free and open-source web browser project, primarily developed and maintained by Google. 25

DirectX (in origine chiamato "Game SDK") è una raccolta di API per lo sviluppo semplificato di videogiochi per sistema operativo Microsoft Windows. Il componente DirectX Graphics permette la presentazione di grafica 2D e 3D a video, direttamente interfacciandosi con la GPU. i, 4, 8, 25

Foreign Function Interface Meccanismo mediante il quale un programma scritto in un linguaggio di programmazione può chiamare routine o fare uso di servizi scritti in un altro. 19

Framerate (FPS) Frequenza di cattura o riproduzione dei fotogrammi che compongono un filmato. Un filmato, o un'animazione al computer, è infatti una sequenza di immagini riprodotte ad una velocità sufficientemente alta da fornire, all'occhio umano, l'illusione del movimento. 5, 32, 33

HMD Head-Mounted Display (in italiano schermo montato sulla testa), schermo montato sulla testa dello spettatore attraverso un casco ad-hoc e può essere monoculare o binoculare. 5

Metal Insieme di API grafiche e di calcolo a basso livello, sviluppate da Apple per offrire accesso diretto alla GPU dei suoi dispositivi. i, 4, 8–11, 16

namespace Set of signs (names) that are used to identify and refer to objects of various kinds. A namespace ensures that all of a given set of objects have unique names so that they can be easily identified. 25

nextInChain Convenzione tipica delle moderne API grafiche derivate da Vulkan, per l'estensione dinamica delle strutture dati. 15, 16

OpenVR API e Runtime che consente accesso alle risorse hardware VR di diversi produttori, senza richiedere conoscenze specifiche dell'hardware stesso. 3

OpenXR Standard aperto che fornisce un set di API per lo sviluppo di applicazioni XR (realtà virtuale, mista, ecc...) compatibili con un grande range di dispositivi AR e VR. i, 3–5, 7, 9, 11, 13, 18–20, 27

Pull Request Contributions to a source code repository that uses a distributed version control system are commonly made by means of a pull request, also known as a merge request. The contributor requests that the project maintainer pull the source code change. 26

Swapchain Serie di framebuffer virtuali utilizzati dalla scheda grafica e dall'API grafica per la stabilizzazione del frame rate, la riduzione delle interruzioni e diversi altri scopi. 19, 20, 26

Vulkan Interfaccia programmatica di applicazione (API) di basso livello, multi-piattaforma in 2D e 3D, annunciata la prima volta al GDC 2015 da Khronos Group. Inizialmente venne presentata come "OpenGL di prossima generazione". i, 4, 8–11, 18–21, 24–26

Sitografia

- [1] Gregg Travers. *WebGPU Fundamentals*. 2020. URL: <https://webgpufundamentals.org> (visitato il 28/05/2026).
- [2] The Khronos Group. *OpenXR Tutorial*. 2024. URL: <https://openxr-tutorial.com> (visitato il 28/05/2026).